In [132…
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
import warnings
warnings.filterwarnings(action = 'ignore')
```

In [133…
```
df = pd.read_csv('housing_data.csv')
```

In [134…
```
df.head()
```

Out[134…

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	396
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	396
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	392
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	394
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	396

In [135…
```
df.shape
```

Out[135…
```
(506, 14)
```

In [136…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     486 non-null    float64
 1   ZN       486 non-null    float64
 2   INDUS    486 non-null    float64
 3   CHAS     486 non-null    float64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      486 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    int64
 9   TAX      506 non-null    int64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    486 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
```

In [137…
```
df.isnull().sum()
```

Out[137…
```
CRIM       20
ZN         20
INDUS      20
CHAS       20
NOX         0
RM          0
AGE        20
DIS         0
RAD         0
TAX         0
PTRATIO     0
B           0
LSTAT      20
MEDV        0
dtype: int64
```

In [138…
```
nc = [feature for feature in df.columns if df[feature].dtype != '0']
```

In [139…
```
nc
```

Out[139…
```
['CRIM',
 'ZN',
 'INDUS',
 'CHAS',
 'NOX',
 'RM',
 'AGE',
 'DIS',
 'RAD',
 'TAX',
 'PTRATIO',
 'B',
 'LSTAT',
 'MEDV']
```

In [140…
```
for i in nc:
    df[i].fillna(df[i].median(), inplace = True)
df
```

Out[140…

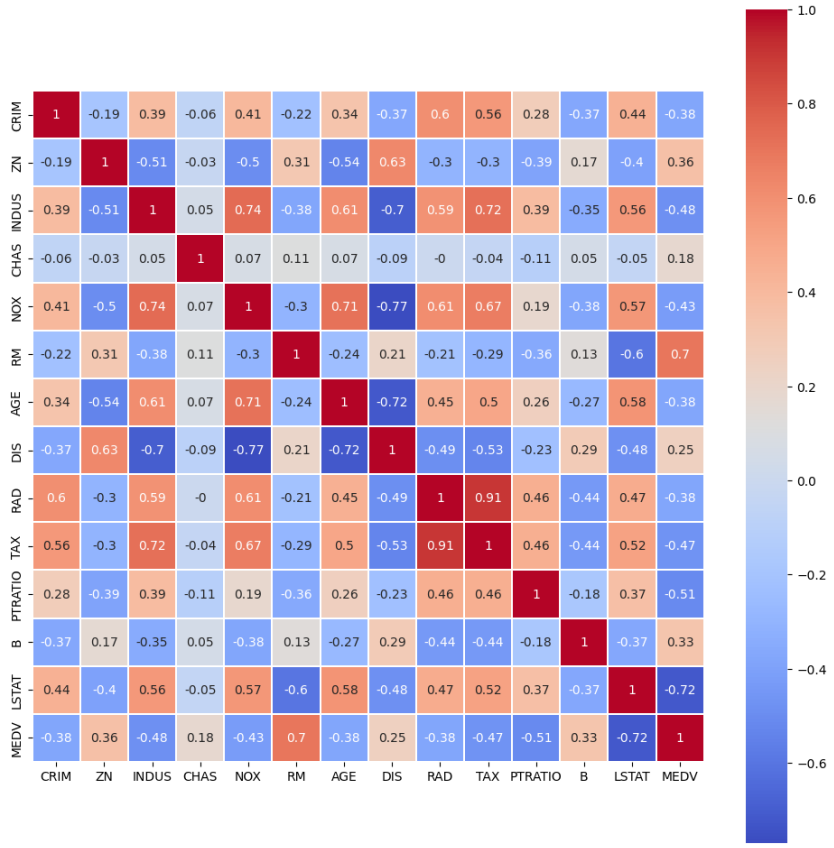
	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	3
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	3
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	3
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	3
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	3
...	...	...	...	...	...	...	...	...	...	...	...	
501	0.06263	0.0	11.93	0.0	0.573	6.593	69.1	2.4786	1	273	21.0	3
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1	273	21.0	3
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1	273	21.0	3
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1	273	21.0	3
505	0.04741	0.0	11.93	0.0	0.573	6.030	76.8	2.5050	1	273	21.0	3

506 rows × 14 columns

In [141…

```
plt.figure(figsize = (12,12))
sb.heatmap(data = df.corr().round(2), annot = True, cmap = 'coolwarm', li
```

Out[141… <Axes: >



In [142…

```
df1 = df[['RM', 'TAX', 'PTRATIO', 'LSTAT', 'MEDV']]
df1
```

Out[142…

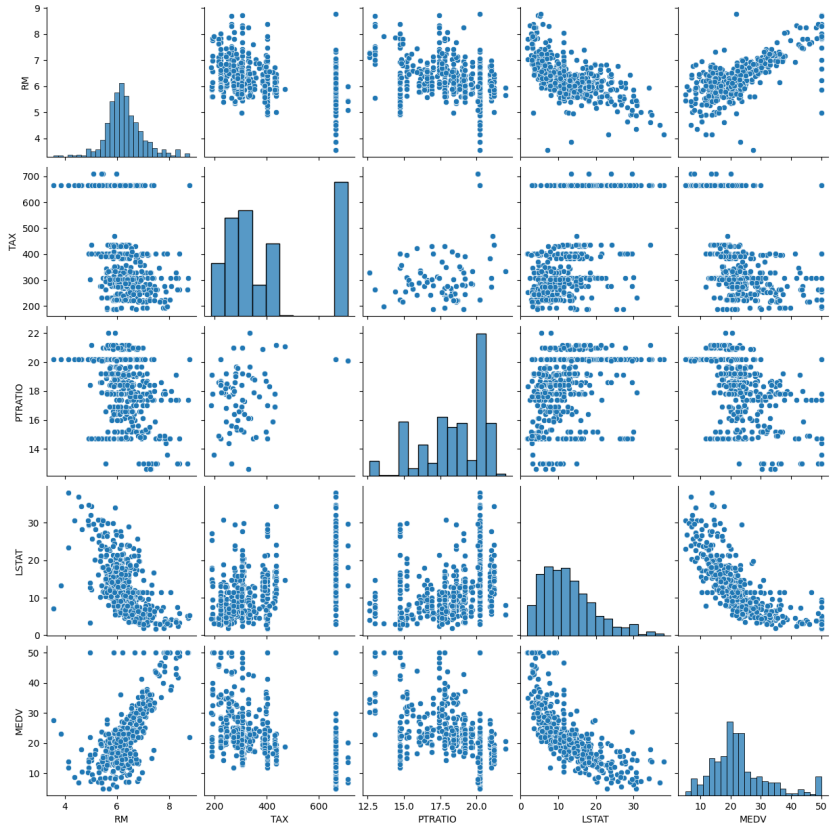
	RM	TAX	PTRATIO	LSTAT	MEDV
0	6.575	296	15.3	4.98	24.0
1	6.421	242	17.8	9.14	21.6
2	7.185	242	17.8	4.03	34.7
3	6.998	222	18.7	2.94	33.4
4	7.147	222	18.7	11.43	36.2
...	...	...	...	...	...
501	6.593	273	21.0	11.43	22.4
502	6.120	273	21.0	9.08	20.6
503	6.976	273	21.0	5.64	23.9
504	6.794	273	21.0	6.48	22.0
505	6.030	273	21.0	7.88	11.9

506 rows × 5 columns

In [143…

```
sb.pairplot(data = df1)
```

Out[143… <seaborn.axisgrid.PairGrid at 0x79e037350100>



In [144…

```
desc = df1.describe().round(2)
desc
```

Out[144…

	RM	TAX	PTRATIO	LSTAT	MEDV
count	506.00	506.00	506.00	506.00	506.00
mean	6.28	408.24	18.46	12.66	22.53
std	0.70	168.54	2.16	7.02	9.20
min	3.56	187.00	12.60	1.73	5.00
25%	5.89	279.00	17.40	7.23	17.02
50%	6.21	330.00	19.05	11.43	21.20
75%	6.62	666.00	20.20	16.57	25.00
max	8.78	711.00	22.00	37.97	50.00

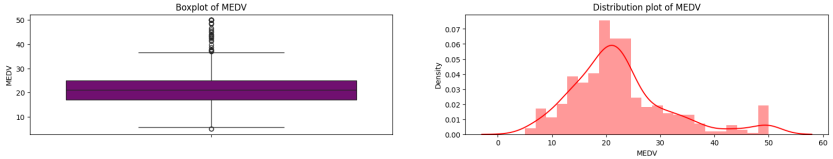
Univariate and Multivariate Analysis

MEDV

In [145…

```
plt.figure(figsize = (20, 3))
plt.subplot(1, 2, 1)
sb.boxplot(df1.MEDV, color = 'purple')
plt.title('Boxplot of MEDV')

plt.subplot(1, 2, 2)
sb.distplot(a = df1.MEDV, color = 'red')
plt.title('Distribution plot of MEDV')
plt.show()
```



In [146…

```
medv_q3 = desc['MEDV']['75%']
medv_q1 = desc['MEDV']['25%']
medv_iqr = medv_q3 - medv_q1
medv_uv = medv_q3 + 1.5 * medv_iqr
medv_lv = medv_q1 - 1.5 * medv_iqr
df1[df1['MEDV'] < medv_lv]
```

Out[146…

	RM	TAX	PTRATIO	LSTAT	MEDV
398	5.453	666	20.2	30.59	5.0
405	5.683	666	20.2	22.98	5.0

In [147…

```
df1[df1['MEDV'] > medv_uv].sort_values(by = ['MEDV', 'RM'])
```

Out[147…

	RM	TAX	PTRATIO	LSTAT	MEDV
190	6.951	398	15.2	5.10	37.0
179	6.980	193	17.8	5.04	37.2
291	7.148	245	19.2	3.56	37.3
226	8.040	307	17.4	11.43	37.6
182	7.155	193	17.8	4.82	37.9
97	8.069	276	18.0	4.21	38.7
180	7.765	193	17.8	7.56	39.8
157	6.943	403	14.7	4.59	41.3
232	8.337	307	17.4	2.47	41.7
202	7.610	348	14.7	3.11	42.3
253	8.259	330	19.1	3.54	42.8
261	7.520	264	13.0	7.26	43.1
268	7.470	264	13.0	3.16	43.5
98	7.820	276	18.0	3.57	43.8
256	7.454	244	15.9	3.11	44.0
224	8.266	307	17.4	4.14	44.8
280	7.820	216	14.9	3.76	45.4
282	7.645	216	14.9	3.01	46.0
228	7.686	307	17.4	11.43	46.7
233	8.247	307	17.4	3.95	48.3
203	7.853	224	14.7	3.81	48.5
262	8.398	264	13.0	5.91	48.8
368	4.970	666	20.2	3.26	50.0
372	5.875	666	20.2	8.88	50.0
371	6.216	666	20.2	9.53	50.0
369	6.683	666	20.2	3.73	50.0
370	7.016	666	20.2	2.96	50.0
161	7.489	403	14.7	1.73	50.0
162	7.802	403	14.7	1.92	50.0
186	7.831	193	17.8	4.45	50.0
195	7.875	255	14.4	2.97	50.0
283	7.923	198	13.6	3.16	50.0
166	7.929	403	14.7	3.70	50.0
204	8.034	224	14.7	2.88	50.0
267	8.297	264	13.0	7.44	50.0

	RM	TAX	PTRATIO	LSTAT	MEDV
163	8.375	403	14.7	3.32	50.0
257	8.704	264	13.0	5.12	50.0
225	8.725	307	17.4	4.63	50.0

In [148…
```
df1.shape
```

Out[148… (506, 5)

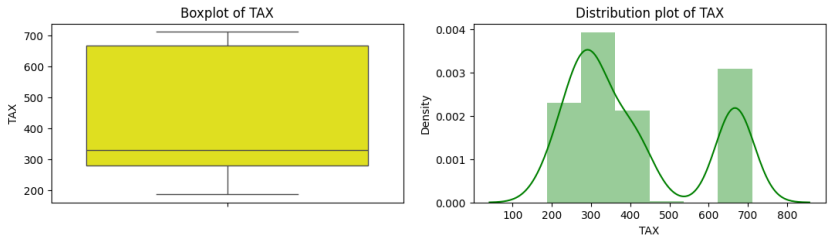
In [149…
```
df2 = df1[~(df1['MEDV'] == 50)]
df2.shape
```

Out[149… (490, 5)

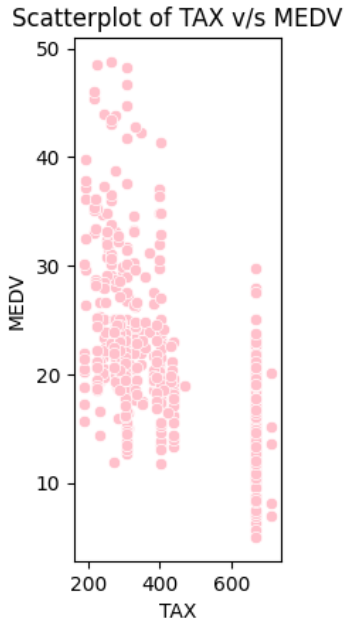
In [150…
```
plt.figure(figsize = (20, 3))
plt.subplot(1, 3, 1)
sb.boxplot(df2.TAX, color = 'yellow')
plt.title('Boxplot of TAX')

plt.subplot(1, 3, 2)
sb.distplot(a = df2.TAX, color = 'green')
plt.title('Distribution plot of TAX')
plt.show()

plt.subplot(1, 3, 3)
sb.scatterplot(x = df2.TAX, y = df2.MEDV, color = 'pink')
plt.title('Scatterplot of TAX v/s MEDV')
```



Out[150… Text(0.5, 1.0, 'Scatterplot of TAX v/s MEDV')



In [151…
```
temp_df = df2[df1['TAX'] > 600].sort_values(by = ['RM', 'MEDV'])
temp_df
```

Out[151…

	RM	TAX	PTRATIO	LSTAT	MEDV
365	3.561	666	20.2	7.12	27.5
367	3.863	666	20.2	13.33	23.1
406	4.138	666	20.2	23.34	11.9
374	4.138	666	20.2	37.97	13.8
384	4.368	666	20.2	30.63	8.8
...	...	...	...	...	...
473	6.980	666	20.2	11.43	29.8
482	7.061	666	20.2	7.01	25.0
375	7.313	666	20.2	13.44	15.0
453	7.393	666	20.2	16.74	17.8
364	8.780	666	20.2	5.29	21.9

132 rows × 5 columns

In [152…
```
temp_df.describe()
```

Out[152…

	RM	TAX	PTRATIO	LSTAT	MEDV
count	132.000000	132.000000	132.000000	132.000000	132.000000
mean	6.000689	667.704545	20.196212	18.828864	14.994697
std	0.712621	8.623365	0.019163	6.590380	5.405825
min	3.561000	666.000000	20.100000	5.290000	5.000000
25%	5.674250	666.000000	20.200000	14.175000	10.900000
50%	6.139500	666.000000	20.200000	17.910000	14.100000
75%	6.407250	666.000000	20.200000	23.052500	19.200000
max	8.780000	711.000000	20.200000	37.970000	29.800000

In [153…

```
TAX_10 = df2[(df2['TAX']<600) & (df2['LSTAT']>=0) & (df2['LSTAT']<10)]['T
TAX_20 = df2[(df2['TAX']<600) & (df2['LSTAT']>=10) & (df2['LSTAT']<20)]['
TAX_30 = df2[(df2['TAX']<600) & (df2['LSTAT']>=20) & (df2['LSTAT']<30)]['
TAX_40 = df2[(df2['TAX']<600) & (df2['LSTAT']>=30)]['TAX'].mean()

indexes = list(df2.index)
for i in indexes:
    if df2['TAX'][i] > 600:
        if (0 <= df2['LSTAT'][i] < 10):
            df2.at[i,'TAX'] = TAX_10
        elif (10 <= df2['LSTAT'][i] < 20):
            df2.at[i,'TAX'] = TAX_20
        elif (20 <= df2['LSTAT'][i] < 30):
            df2.at[i,'TAX'] = TAX_30
        elif (df2['LSTAT'][i] >30):
            df2.at[i,'TAX'] = TAX_40

print('Values imputed successfully')
```

```
Values imputed successfully
```

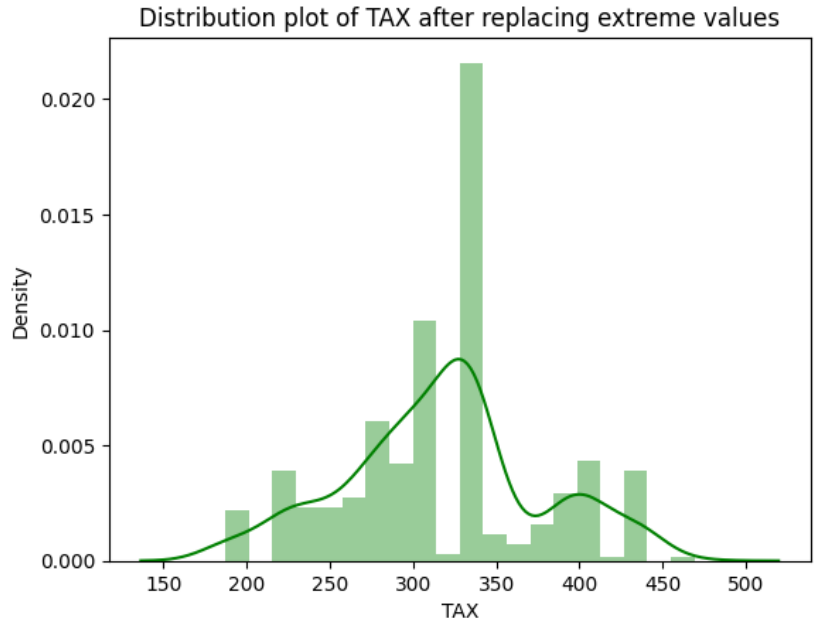
In [154…

```
df2[df2['TAX'] > 600]['TAX'].count()
```

Out[154… 0

In [155…

```
sb.distplot(a = df2.TAX, color = 'green')
plt.title('Distribution plot of TAX after replacing extreme values')
plt.show()
```

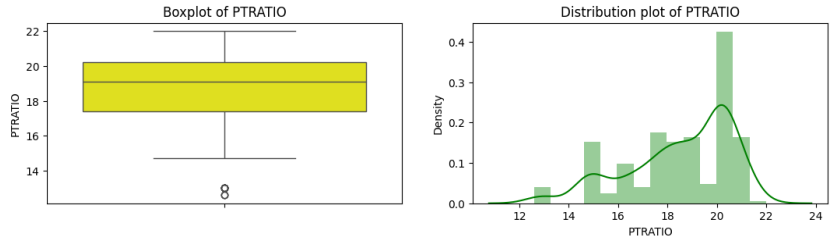


In [156…

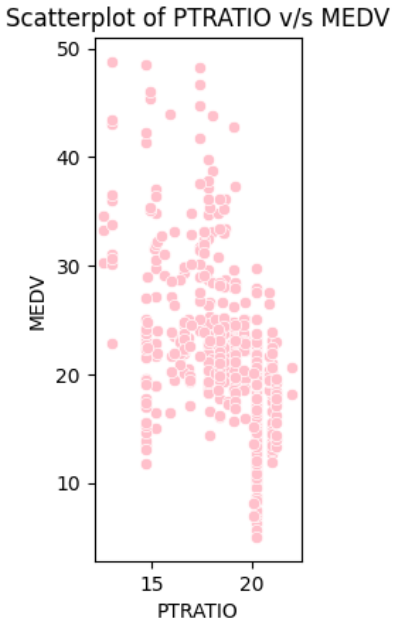
```
plt.figure(figsize = (20, 3))
plt.subplot(1, 3, 1)
sb.boxplot(df2.PTRATIO, color = 'yellow')
plt.title('Boxplot of PTRATIO')

plt.subplot(1, 3, 2)
sb.distplot(a = df2.PTRATIO, color = 'green')
plt.title('Distribution plot of PTRATIO')
plt.show()

plt.subplot(1, 3, 3)
sb.scatterplot(x = df2.PTRATIO, y = df2.MEDV, color = 'pink')
plt.title('Scatterplot of PTRATIO v/s MEDV')
```



Out[156… Text(0.5, 1.0, 'Scatterplot of PTRATIO v/s MEDV')



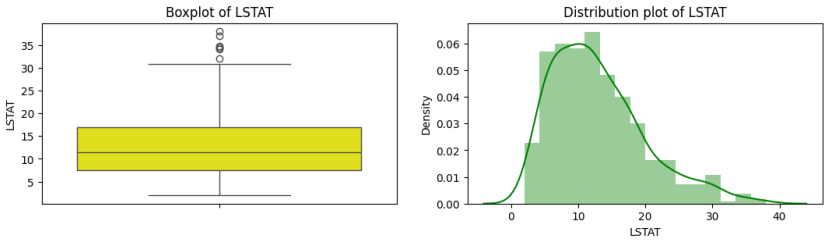
In [157…
```
df2[df2['PTRATIO'] < 14].sort_values(by = ['LSTAT', 'MEDV'])
```

Out[157…

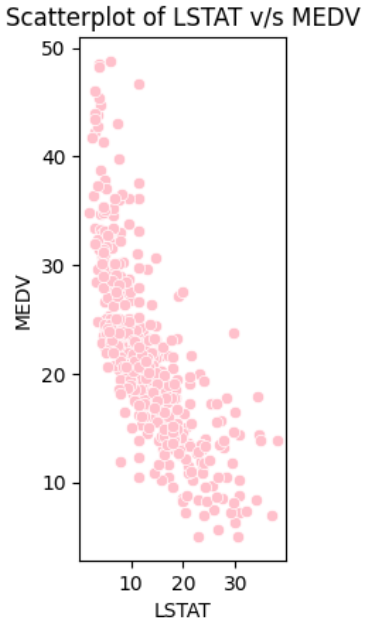
	RM	TAX	PTRATIO	LSTAT	MEDV
268	7.470	264.0	13.0	3.16	43.5
196	7.287	329.0	12.6	4.08	33.3
262	8.398	264.0	13.0	5.91	48.8
198	7.274	329.0	12.6	6.62	34.6
259	6.842	264.0	13.0	6.90	30.1
261	7.520	264.0	13.0	7.26	43.1
258	7.333	264.0	13.0	7.79	36.0
264	7.206	264.0	13.0	8.10	36.5
197	7.107	329.0	12.6	8.61	30.3
260	7.203	264.0	13.0	9.59	33.8
265	5.560	264.0	13.0	10.45	22.8
263	7.327	264.0	13.0	11.25	31.0
266	7.014	264.0	13.0	14.79	30.7

In [158…
```
plt.figure(figsize = (20, 3))
plt.subplot(1, 3, 1)
sb.boxplot(df2.LSTAT, color = 'yellow')
plt.title('Boxplot of LSTAT')
plt.subplot(1, 3, 2)
sb.distplot(a = df2.LSTAT, color = 'green')
plt.title('Distribution plot of LSTAT')
plt.show()

plt.subplot(1, 3, 3)
sb.scatterplot(x = df2.LSTAT, y = df2.MEDV, color = 'pink')
plt.title('Scatterplot of LSTAT v/s MEDV')
```



Out[158… Text(0.5, 1.0, 'Scatterplot of LSTAT v/s MEDV')



In [159…
```
lstat_q3 = desc['LSTAT']['75%']
lstat_q1 = desc['LSTAT']['25%']
lstat_iqr = lstat_q3 - lstat_q1
lstat_uv = lstat_q3 + 1.5 * lstat_iqr
lstat_lv = lstat_q1 - 1.5 * lstat_iqr
df1[df1['LSTAT'] > lstat_uv].sort_values(by='LSTAT')
```

Out[159…

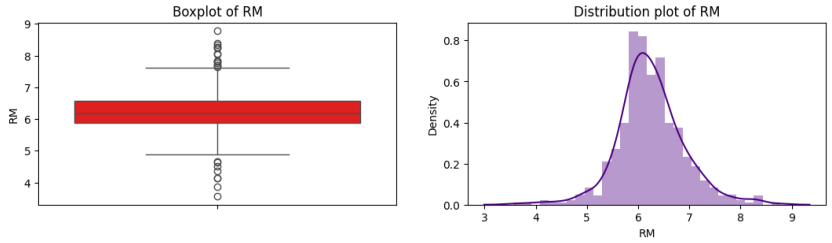
	RM	TAX	PTRATIO	LSTAT	MEDV
398	5.453	666	20.2	30.59	5.0
388	4.880	666	20.2	30.62	10.2
384	4.368	666	20.2	30.63	8.8
48	5.399	233	17.9	30.81	14.4
385	5.277	666	20.2	30.81	7.2
387	5.000	666	20.2	31.99	7.4
438	5.935	666	20.2	34.02	8.4
412	4.628	666	20.2	34.37	17.9
141	5.019	437	21.2	34.41	14.4
373	4.906	666	20.2	34.77	13.8
414	4.519	666	20.2	36.98	7.0
374	4.138	666	20.2	37.97	13.8

In [160…

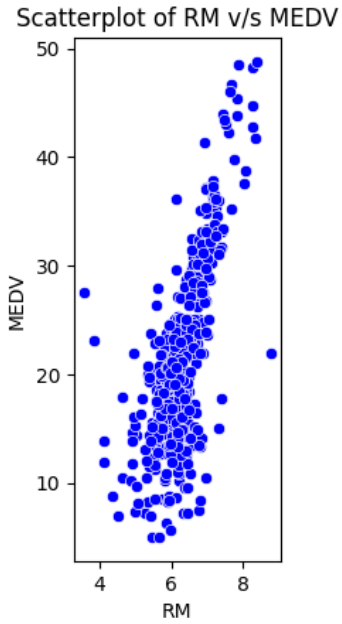
```
plt.figure(figsize = (20, 3))
plt.subplot(1, 3, 1)
sb.boxplot(df2.RM, color = 'RED')
plt.title('Boxplot of RM')

plt.subplot(1, 3, 2)
sb.distplot(a = df2.RM, color = 'INDIGO')
plt.title('Distribution plot of RM')
plt.show()

plt.subplot(1, 3, 3)
sb.scatterplot(x = df2.RM, y = df2.MEDV, color = 'BLUE')
plt.title('Scatterplot of RM v/s MEDV')
```



Out[160… Text(0.5, 1.0, 'Scatterplot of RM v/s MEDV')



In [161…

```
rm_q3 = desc['RM']['75%']
rm_q1 = desc['RM']['25%']
rm_iqr = rm_q3 - rm_q1
rm_uv = rm_q3 + 1.5 * rm_iqr
rm_lv = rm_q1 - 1.5 * rm_iqr
df1[df1['RM'] > rm_uv].sort_values(by=['RM','MEDV'])
```

Out[161…

	RM	TAX	PTRATIO	LSTAT	MEDV
180	7.765	193	17.8	7.56	39.8
162	7.802	403	14.7	1.92	50.0
98	7.820	276	18.0	3.57	43.8
280	7.820	216	14.9	3.76	45.4
186	7.831	193	17.8	4.45	50.0
203	7.853	224	14.7	3.81	48.5
195	7.875	255	14.4	2.97	50.0
283	7.923	198	13.6	3.16	50.0
166	7.929	403	14.7	3.70	50.0
204	8.034	224	14.7	2.88	50.0
226	8.040	307	17.4	11.43	37.6
97	8.069	276	18.0	4.21	38.7
233	8.247	307	17.4	3.95	48.3
253	8.259	330	19.1	3.54	42.8
224	8.266	307	17.4	4.14	44.8
267	8.297	264	13.0	7.44	50.0
232	8.337	307	17.4	2.47	41.7
163	8.375	403	14.7	3.32	50.0
262	8.398	264	13.0	5.91	48.8
257	8.704	264	13.0	5.12	50.0
225	8.725	307	17.4	4.63	50.0
364	8.780	666	20.2	5.29	21.9

In [162…

```
rm_q3 = desc['RM']['75%']
rm_q1 = desc['RM']['25%']
rm_iqr = rm_q3 - rm_q1
rm_uv = rm_q3 + 1.5 * rm_iqr
rm_lv = rm_q1 - 1.5 * rm_iqr
df1[df1['RM'] < rm_lv].sort_values(by=['RM','MEDV'])
```

Out[162…

	RM	TAX	PTRATIO	LSTAT	MEDV
365	3.561	666	20.2	7.12	27.5
367	3.863	666	20.2	13.33	23.1
406	4.138	666	20.2	23.34	11.9
374	4.138	666	20.2	37.97	13.8
384	4.368	666	20.2	30.63	8.8
414	4.519	666	20.2	36.98	7.0
412	4.628	666	20.2	34.37	17.9
386	4.652	666	20.2	28.28	10.5

In [163…

```
df2.shape
```

Out[163… (490, 5)

In [164…

```
df3=df2.drop(axis=0,index=[365,367,364])
```

In [165…

```
df3.shape
```

Out[165… (487, 5)

In [166…

```
X = df3.iloc[:,0:4].values
y = df3.iloc[:,-1:].values
```

In [167…

```
print(f"Shape of Dependent Variable X = {X.shape}")
print(f"Shape of Independent Variable y = {y.shape}")
```

```
Shape of Dependent Variable X = (487, 4)
Shape of Independent Variable y = (487, 1)
```

In [168…

```
def FeatureScaling(X):
    mean = np.mean(X,axis=0)
    std = np.std(X,axis=0)
    for i in range(X.shape[1]):
        X[:,i] = (X[:,i]-mean[i])/std[i]
    return X
```

In [169…

```
X = FeatureScaling(X)
```

In [170…

```
X.shape
```

Out[170… (487, 4)

In [171…

```
m,n = X.shape
X = np.append(arr=np.ones((m,1)),values=X,axis=1)
```

In [172…

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,ran
```

In [173…

```
print(f"Shape of X_train = {X_train.shape}")
print(f"Shape of X_test = {X_test.shape}")
print(f"Shape of y_train = {y_train.shape}")
```

```
print(f"Shape of y_test = {y_test.shape}")
```

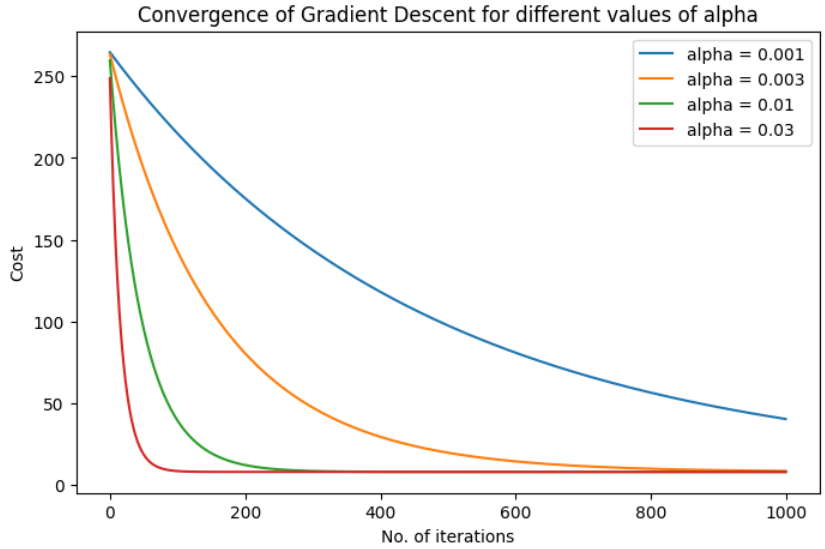
```
Shape of X_train = (389, 5)
Shape of X_test = (98, 5)
Shape of y_train = (389, 1)
Shape of y_test = (98, 1)
```

In [174…
```
def ComputeCost(X,y,theta):
    m=X.shape[0] #number of data points in the set
    J = (1/(2*m)) * np.sum((X.dot(theta) - y)**2)
    return J
```

In [175…
```
def GradientDescent(X,y,theta,alpha,no_of_iters):
    m=X.shape[0]
    J_Cost = []
    for i in range(no_of_iters):
        error = np.dot(X.transpose(),(X.dot(theta)-y))
        theta = theta - alpha * (1/m) * error
        J_Cost.append(ComputeCost(X,y,theta))
    return theta, np.array(J_Cost)
```

In [176…
```
iters = 1000
alpha1 = 0.001
theta1 = np.zeros((X_train.shape[1],1))
theta1, J_Costs1 = GradientDescent(X_train,y_train,theta1,alpha1,iters)
alpha2 = 0.003
theta2 = np.zeros((X_train.shape[1],1))
theta2, J_Costs2 = GradientDescent(X_train,y_train,theta2,alpha2,iters)
alpha3 = 0.01
theta3 = np.zeros((X_train.shape[1],1))
theta3, J_Costs3 = GradientDescent(X_train,y_train,theta3,alpha3,iters)
alpha4 = 0.03
theta4 = np.zeros((X_train.shape[1],1))
theta4, J_Costs4 = GradientDescent(X_train,y_train,theta4,alpha4,iters)
```

In [177…
```
plt.figure(figsize=(8,5))
plt.plot(J_Costs1,label = 'alpha = 0.001')
plt.plot(J_Costs2,label = 'alpha = 0.003')
plt.plot(J_Costs3,label = 'alpha = 0.01')
plt.plot(J_Costs4,label = 'alpha = 0.03')
plt.title('Convergence of Gradient Descent for different values of alpha'
plt.xlabel('No. of iterations')
plt.ylabel('Cost')
plt.legend()
plt.show()
```



In [178…
```
theta4
```

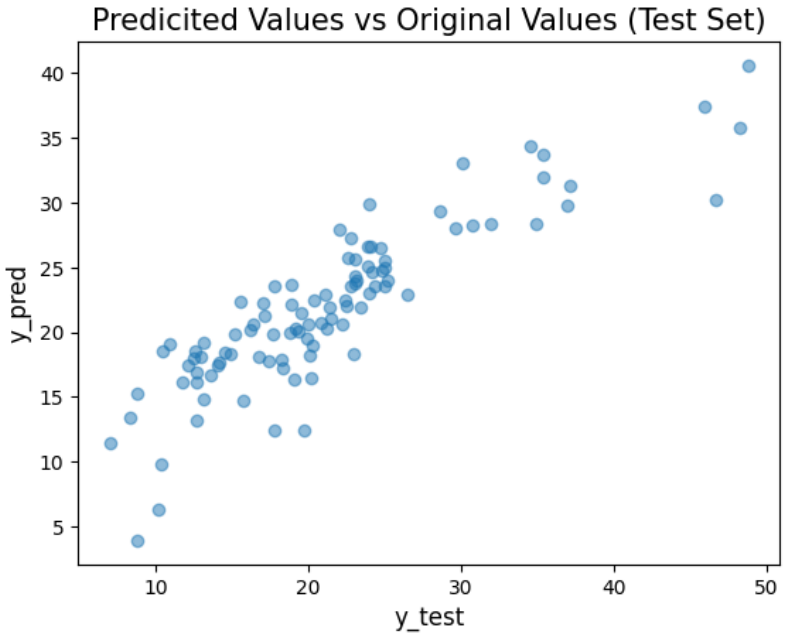
Out[178…
```
array([[21.73546011],
       [ 2.86076194],
       [-1.078047  ],
       [-1.96051041],
       [-2.87113215]])
```

In [179…
```
def Predict(X,theta):
    y_pred = X.dot(theta)
    return y_pred
```

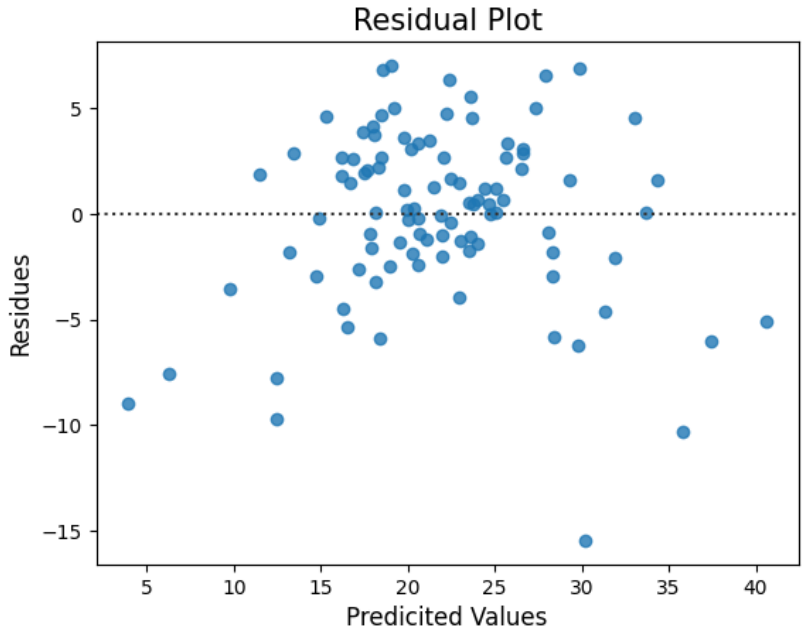
In [180…
```
y_pred=Predict(X_test,theta4)
y_pred[:5]
```

Out[180…
```
array([[23.54777745],
       [28.09399088],
       [16.16554384],
       [18.97965458],
       [17.66976105]])
```

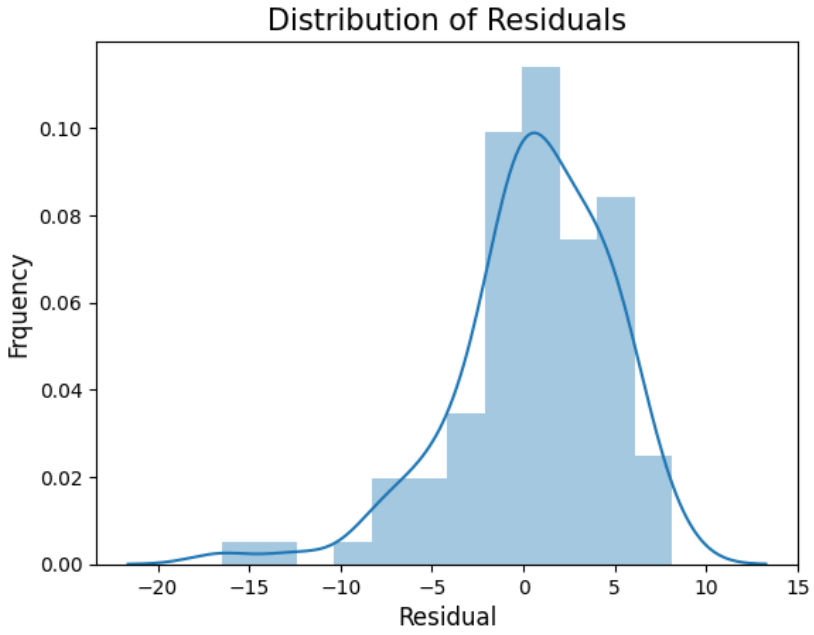
In [181…
```
plt.scatter(x=y_test,y=y_pred,alpha=0.5)
plt.xlabel('y_test',size=12)
plt.ylabel('y_pred',size=12)
plt.title('Predicited Values vs Original Values (Test Set)',size=15)
plt.show()
```



In [182…
```
sb.residplot(x=y_pred,y=y_pred-y_test)
plt.xlabel('Predicited Values',size=12)
plt.ylabel("Residues",size=12)
plt.title('Residual Plot',size=15)
plt.show()
```



In [183…
```
sb.distplot(y_pred-y_test)
plt.xlabel('Residual',size=12)
plt.ylabel('Frquency',size=12)
plt.title('Distribution of Residuals',size=15)
plt.show()
```



In [184…
```
from sklearn import metrics
r2= metrics.r2_score(y_test,y_pred)
N,p = X_test.shape
adj_r2 = 1-((1-r2)*(N-1))/(N-p-1)
print(f'R^2 = {r2}')
print(f'Adjusted R^2 = {adj_r2}')
```

```
R^2 = 0.7467879874493435
Adjusted R^2 = 0.7330264650281122
```

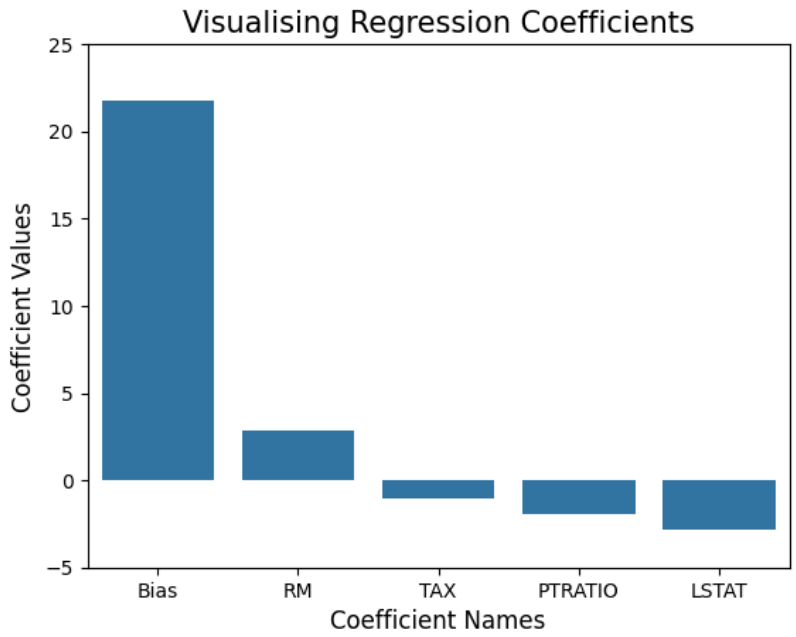
In [185…
```
from sklearn import metrics
mse = metrics.mean_squared_error(y_test,y_pred)
mae = metrics.mean_absolute_error(y_test,y_pred)
rmse = np.sqrt(metrics.mean_squared_error(y_test,y_pred))
print(f'Mean Squared Error: {mse}',f'Mean Absolute Error: {mae}',f'Root M
```

```
Mean Squared Error: 18.27759183945454
Mean Absolute Error: 3.2403659815833694
Root Mean Squared Error: 4.275230033513348
```

In [186…
```
coeff=np.array([y for x in theta4 for y in x]).round(2)
features=['Bias','RM','TAX','PTRATIO','LSTAT']
eqn = 'MEDV = '
for f,c in zip(features,coeff):
    eqn+=f" + ({c} * {f})";
print(eqn)
```

```
MEDV =  + (21.74 * Bias) + (2.86 * RM) + (-1.08 * TAX) + (-1.96 * PTRATIO)
+ (-2.87 * LSTAT)
```

In [187…
```
sb.barplot(x=features,y=coeff)
plt.ylim([-5,25])
plt.xlabel('Coefficient Names',size=12)
plt.ylabel('Coefficient Values',size=12)
plt.title('Visualising Regression Coefficients',size=15)
plt.show()
```



In [ ]: